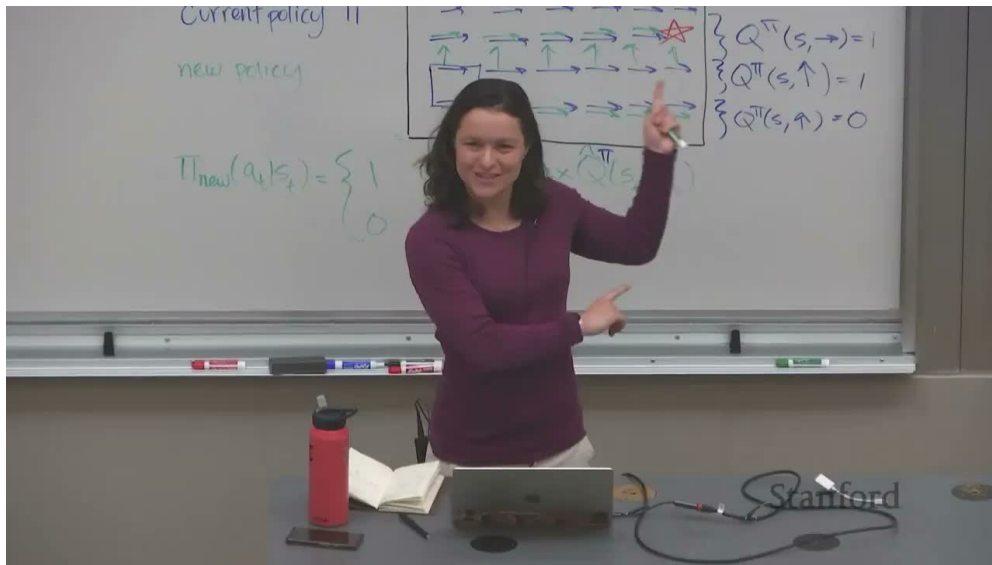


课程笔记

CS224R 第 6 讲: Q-Learning

五道口纳什 & Codex

2025-04-18



视频作者/频道: Stanford CS224R

发布日期: 2025-04-18

视频时长: 01:01:35

视频链接: <https://www.youtube.com/watch?v=-7kv6jf0isQ>

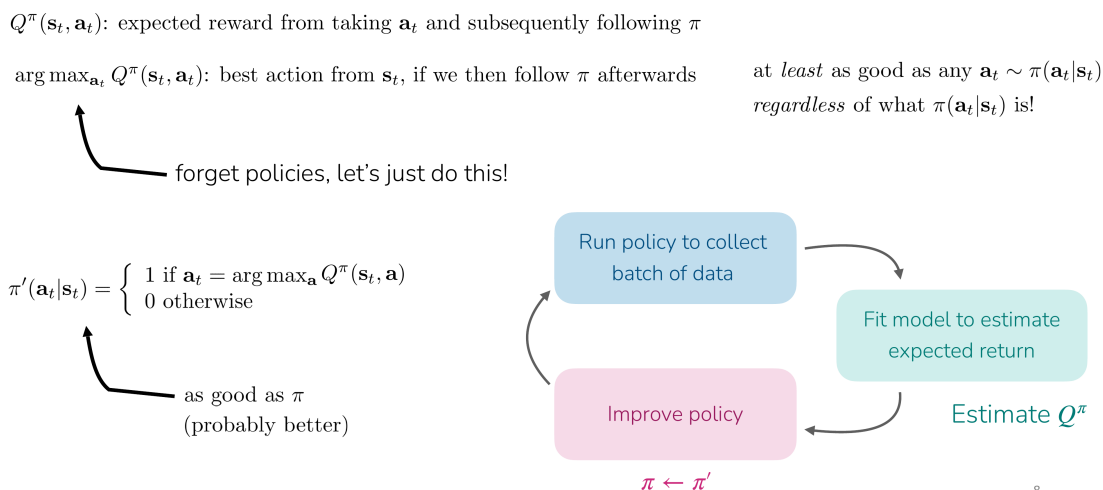
目录

1 从 policy improvement 到只学 Q: Q-learning 的思想跳跃	2
1.1 为什么这一步成立	2
1.2 本章小结	2
2 Bellman optimality: Q-learning 的目标究竟在拟合什么	3
2.1 单步目标	3
2.2 off-policy 数据为何天然可用	3
2.3 本章小结	3
3 Q-learning 的实践算法: 数据收集、replay buffer 与 target network	4
3.1 一个基础训练循环	4
3.2 为什么要引入 target network	4
3.3 本章小结	5
4 进一步稳定化: Double Q-learning 与 N-step returns	5
4.1 Double Q-learning: 把“选动作”和“评动作”拆开	5
4.2 N-step returns: 更快把奖励向前传播	6
4.3 什么时候该用哪种 online RL 算法	6
4.4 本章小结	6
5 总结与延伸	6

1 从 policy improvement 到只学 Q: Q-learning 的思想跳跃

本讲的起点，是对上一讲 off-policy actor-critic 的进一步极简化。既然我们已经知道 $Q^\pi(s, a)$ 描述“在状态 s 采取动作 a 后，再按策略 π 行动的未来回报”，那是否可以直接在每个状态选择 Q 最大的动作，而不再单独维护一个显式 actor？

Can we omit policy gradient completely?



Slide adapted from Sergey Levine

8

图 1: 从显式 policy gradient 过渡到直接对 Q 做 greedy improvement

讲者给出的答案是肯定的。若我们已经有准确的 $Q^\pi(s, a)$ ，那么定义

$$\pi'(a_t | s_t) = \begin{cases} 1, & a_t = \arg \max_a Q^\pi(s_t, a), \\ 0, & \text{otherwise,} \end{cases}$$

则新策略至少不会比旧策略更差，往往还会更好。这里的思想与 policy iteration 一致：先评估，再贪心改进。

1.1 为什么这一步成立

原因在于 $Q^\pi(s, a)$ 本身已经告诉我们“在该状态下，先做哪个动作最赚”。既然如此，就没必要再通过一个独立 actor 去学习同一件事。对离散动作空间尤其如此，枚举所有动作的 Q 值并取最大值往往更直接。

Q-learning 的哲学

actor-critic 里，actor 负责把“好坏估计”翻译成策略；Q-learning 里，这个翻译过程被一个 $\arg \max$ 直接替代了。

1.2 本章小结

本节完成了最关键的概念跨越：从“critic 辅助 actor”转向“critic 独自完成策略改进”。这就是 value-based RL 的核心立场。

2 Bellman optimality: Q-learning 的目标究竟在拟合什么

如果只剩下 Q 网络，我们需要一个自洽的目标来训练它。讲者先回顾任意策略下的 Bellman equation：

$$Q^\pi(s, a) = r(s, a) + \gamma \mathbb{E}_{s' \sim p(\cdot|s, a), a' \sim \pi(\cdot|s')} [Q^\pi(s', a')].$$

若把策略换成最优策略 π^* ，就得到 Bellman optimality equation：

$$Q^{\pi^*}(s, a) = r(s, a) + \gamma \mathbb{E}_{s' \sim p(\cdot|s, a)} \left[\max_{a'} Q^{\pi^*}(s', a') \right].$$

Q-learning 的全部训练逻辑，就是让一个参数化函数 $\hat{Q}_\phi$ 尽量满足这个方程。

From actor-critic to critic only

Q-learning

1. take action $\mathbf{a} \sim \pi(\mathbf{a}|\mathbf{s})$, get $(\mathbf{s}, \mathbf{a}, \mathbf{s}', r)$, store in $\mathcal{R}$
2. sample a batch $\{\mathbf{s}_i, \mathbf{a}_i, r_i, \mathbf{s}'_i\}$ from buffer $\mathcal{R}$
3. update $\hat{Q}_\phi$ using targets $y_i = r_i + \gamma \max_{\mathbf{a}'} \hat{Q}_\phi(\mathbf{s}'_i, \mathbf{a}')$
4. define new policy $\pi(\mathbf{a}_t|\mathbf{s}_t) = \begin{cases} 1 & \text{if } \mathbf{a}_t = \arg \max_{\mathbf{a}} \hat{Q}_\phi(\mathbf{s}_t, \mathbf{a}) \\ 0 & \text{otherwise} \end{cases}$

Terminology

Bellman equation

Bellman optimality equation

Why does make sense?

Recall: $Q^\pi(\mathbf{s}, \mathbf{a}) = r(\mathbf{s}, \mathbf{a}) + \gamma \mathbb{E}_{s' \sim p(\cdot|\mathbf{s}, \mathbf{a}), \bar{\mathbf{a}}' \sim \pi(\cdot|\mathbf{s}')} [Q^\pi(\mathbf{s}', \bar{\mathbf{a}}')]$ for all $(\mathbf{s}, \mathbf{a})$

-> This holds for any policy π (including the optimal policy π^*)

If π^* is the optimal policy, we also get:

$$Q^{\pi^*}(\mathbf{s}, \mathbf{a}) = r(\mathbf{s}, \mathbf{a}) + \gamma \mathbb{E}_{s' \sim p(\cdot|\mathbf{s}, \mathbf{a})} \left[\max_{\bar{\mathbf{a}}'} Q^{\pi^*}(\mathbf{s}', \bar{\mathbf{a}}') \right] \text{ for all } (\mathbf{s}, \mathbf{a})$$

When we optimize step 3, we are trying to make this equation hold!
 the action π^* would take in state $\mathbf{s}'$

12

图 2: 从 actor-critic 退化到 critic-only 之后，Bellman optimality 成为 Q-learning 的监督目标

2.1 单步目标

对 replay buffer 中的一条 transition (s_i, a_i, r_i, s'_i) ，Q-learning 构造目标

$$y_i = r_i + \gamma \max_{a'} \hat{Q}_\phi(s'_i, a').$$

然后把 $\hat{Q}_\phi(s_i, a_i)$ 回归到 y_i 。这一步像监督学习，但标签又由当前 Q 网络自己间接生成，因此它是典型的 bootstrap learning。

2.2 off-policy 数据为何天然可用

Q-learning 的一个突出优点是天然 off-policy。因为目标只要求知道当前 transition 的 s, a, r, s' ，以及下一个状态上“当前 Q 认为最优的动作值”，并不要求数据里的动作来自当前 greedy policy。于是训练数据可以来自任何探索策略。

2.3 本章小结

Bellman optimality equation 给了 Q-learning 一个非常强的训练对象：不是直接模仿某个策略，而是直接逼近最优动作价值函数。一旦这个函数足够准，greedy policy 自然浮现出来。

3 Q-learning 的实践算法: 数据收集、replay buffer 与 target network

虽然公式优雅, 但直接套用 Q-learning 往往不稳定。原因很直观: 目标值里也用到了当前网络自己的预测, 于是标签在优化过程中不断变化, 形成 moving target。

3.1 一个基础训练循环

讲者给出的 practical loop 可以概括为:

1. 用某个探索策略收集 transition (s, a, s', r) , 存入 replay buffer $\mathcal{R}$;
2. 从 $\mathcal{R}$ 中采样 mini-batch;
3. 构造目标 $y_i = r_i + \gamma \max_{a'} \hat{Q}_{\phi'}(s'_i, a')$;
4. 最小化平方损失

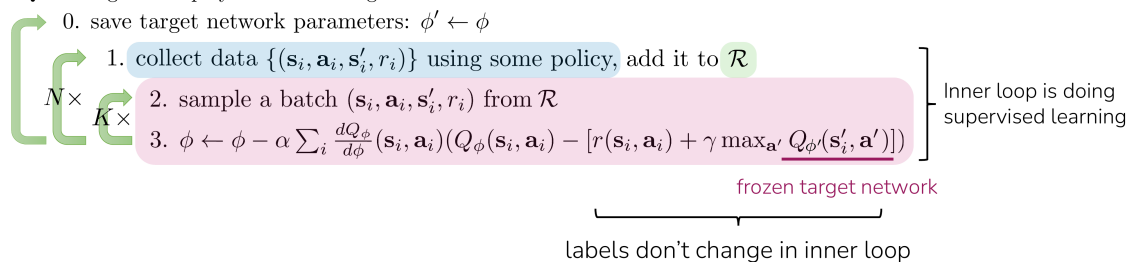
$$\mathcal{L}(\phi) = \frac{1}{2} \sum_i \|\hat{Q}_{\phi}(s_i, a_i) - y_i\|^2.$$

3.2 为什么要引入 target network

若直接令 $\phi' = \phi$, 那么每次参数更新都会同时改变预测值和标签, 导致优化极不稳定。DQN 的关键修复, 是冻结一份 target network 参数 ϕ' , 只隔一段时间才从在线网络 ϕ 复制一次。

How to make Q-learning stable?

Q-learning with replay buffer and target network:



Corresponds to DQN algorithm ("deep Q network")

Mnih et al. *Playing Atari with Deep Reinforcement Learning*. 2013.

Along with an actor-critic method, you'll implement this algorithm in HW 2!

18

图 3: target network 通过冻结标签来源, 缓解 moving target 问题

这带来的收益非常直接: 在内层若干步更新中, 监督目标不再剧烈漂移, 优化更像标准 supervised learning。

DQN 的两个核心工程件

replay buffer 负责打乱相关性、复用历史数据；target network 负责减慢标签变化速度。少了任何一个，深度 Q-learning 都更容易发散。

3.3 本章小结

Q-learning 之所以能在深度网络场景下真正跑起来，不只是因为 Bellman 公式，更因为 replay buffer 和 target network 这两个工程机制把 bootstrap 学习的数值不稳定压了下去。

4 进一步稳定化：Double Q-learning 与 N-step returns

即便有 target network, Q-learning 仍有两个常见问题：一是 max 操作倾向于高估 noisy value；二是一步 bootstrap 在早期容易偏差太大、学习传播过慢。

4.1 Double Q-learning：把“选动作”和“评动作”拆开

标准 Q-learning 中，

$$\max_{a'} Q_{\phi'}(s', a') = Q_{\phi'}(s', \arg \max_{a'} Q_{\phi'}(s', a')),$$

同一个网络既负责选最大动作，也负责给这个动作打分，因此会系统性偏向高估。

Double Q-learning

note that $\max_{a'} Q_{\phi'}(s', a') = Q_{\phi'}(s', \arg \max_{a'} Q_{\phi'}(s', a'))$

value also comes from $Q_{\phi'}$ action selected according to $Q_{\phi'}$

if the noise in these is decorrelated, the problem goes away!

idea: don't use the same network to choose the action and evaluate value!

“double” Q-learning: use two networks:

$$Q_{\phi_A}(s, a) \leftarrow r + \gamma Q_{\phi_B}(s', \arg \max_{a'} Q_{\phi_A}(s', a'))$$

$$Q_{\phi_B}(s, a) \leftarrow r + \gamma Q_{\phi_A}(s', \arg \max_{a'} Q_{\phi_B}(s', a'))$$

if the two Q's are noisy in *different* ways, there is no problem

Slide adapted from Sergey Levine

22

图 4: Double Q-learning 用两个网络分离动作选择与动作评价，减少高估偏差

Double Q-learning 的思路是：用 Q_{ϕ_A} 负责选择动作，再用 Q_{ϕ_B} 评估这个动作的价值，反之亦然。只要两者噪声不完全相关，高估就会明显减轻。

4.2 N-step returns: 更快把奖励向前传播

一步目标

$$y_{j,t} = r_{j,t} + \gamma \max_{a_{j,t+1}} Q_{\phi'}(s_{j,t+1}, a_{j,t+1})$$

在 Q 值估计很差时，几乎完全依赖即时奖励，因此回报信息传播慢。于是可以构造 N-step target:

$$y_{j,t}^{(N)} = \sum_{t'=t}^{t+N-1} \gamma^{t'-t} r_{j,t'} + \gamma^N \max_{a_{j,t+N}} Q_{\phi'}(s_{j,t+N}, a_{j,t+N}).$$

N-step returns

Q-learning target: $y_{j,t} = r_{j,t} + \gamma \max_{a_{j,t+1}} Q_{\phi'}(s_{j,t+1}, a_{j,t+1})$

these are the only values that matter if $Q_{\phi'}$ is bad!

these values are important if $Q_{\phi'}$ is good

Can we construct these multi-step targets for Q-learning?

N-step target: $y_{j,t} = \sum_{t'=t}^{t+N-1} \gamma^{t-t'} r_{j,t'} + \gamma^N \max_{a_{j,t+N}} Q_{\phi'}(s_{j,t+N}, a_{j,t+N})$

these are the rewards for
policy that collected the data

+ (far) less biased target values when Q-values are inaccurate

+ typically faster learning, especially early on

- only actually correct when learning on-policy

(not an issue when $N=1$!)

Ways to fix? Most commonly: ignore the problem & still use $N > 1$

Can also:

- dynamically choose N to only use data that follows current policy
(if data mostly on-policy, action space is small)
- use importance sampling

25

图 5: N-step return 用更多真实奖励替代短视 bootstrap，但在 off-policy 下会引入额外偏差

它的优点是偏差更小、奖励传播更快，尤其训练早期常常显著加速学习；但问题也很明确：中间这 N 步奖励来自收集数据的行为策略，因此严格来说只在 on-policy 情况下完全一致。实践中很多系统仍然使用 $N > 1$ ，接受这部分近似。

4.3 什么时候该用哪种 online RL 算法

讲者最后做了一个简短但非常实用的总结：若重视稳定性和易用性，常选 PPO；若动作空间离散且希望直接学习 value，可选 DQN 及其变体；若更在乎数据效率，则 SAC 及其变体往往更合适。

4.4 本章小结

本节展示了深度 Q-learning 为什么不是“Bellman 方程 + SGD”这么简单。Double Q-learning 解决高估偏差，N-step return 解决信息传播慢，两者都是为了让 value-based RL 在真实网络训练里更靠谱。

5 总结与延伸

这一讲把 online RL 的最后一块主拼图补上了。前几讲从 policy gradient 到 actor-critic，核心都还是“显式维护策略，再想办法更新它”；而 Q-learning 走的是另一条路：只要把每个状态动作

对的价值学准，策略本身就可以由 greedy rule 自动产生。

从理论上讲，本讲最关键的是 Bellman optimality equation，它把“最优策略”变成“最优 Q 函数的自洽条件”。从工程上看，最关键的是 replay buffer、target network、Double Q-learning 与 N-step return，这些机制共同让深度 value learning 不至于在 bootstrap 自举过程中失控。

更大的课程脉络也在这里收束起来了。PPO、SAC、DQN 各自代表在线强化学习的三种典型设计取向：稳定优先、数据效率优先、离散动作 value-based 优先。后续讲 offline RL 和 language model RL 时，很多方法仍然是在这三条主线的基础上做延伸。